Risk-Based Test Strategy and Coverage Plan

Insurance Claims Processing Platform (Auto, Health, Property)

Prepared by: Arul Amirtharaj | October 2026

1. Executive Summary

The claims platform tests every feature to the same depth. Half of production incidents come from two areas, claims calculation and third-party integrations, and those are exactly the areas with no test automation. Regression testing is manual, its coverage is unknown, and there are no shared quality metrics. As a result, the riskiest code gets the same attention as the least risky code, and nobody can say with evidence whether a release is safe.

This strategy rests on three moves:

1. **Test in proportion to risk.** Score every module and integration on likelihood and impact, place it in one of three tiers, and set test depth, automation targets and release gates by tier.
2. **Close the automation gap where incidents originate.** Build data-driven calculation tests, consumer-driven contract tests and service virtualization for integrations first, before investing in UI automation.
3. **Make quality visible.** Link requirements and business rules to tests so coverage can be measured, and track a small set of metrics for claims accuracy, integration reliability and customer experience, enforced through release gates.

> **Targets after two quarters:** a 50% reduction in calculation and integration incidents; at least 90% of Tier 1 business rules covered by automated tests; the bi-weekly regression cycle cut from days of manual effort to under 4 hours, mostly automated; and a single quality dashboard used in every release decision.

2. Current State and Assumptions

2.1 What the problem statement tells us

Observation	Consequence	Response in this strategy
50% of incidents are calculation errors or integration failures	Quality effort is not aimed where failures happen	Both areas rated Tier 1; they get automation first (Sections 4.1, 4.2, 6)
Automation only for platform services	Product modules and integrations rely on slow, manual checks	Automate product calculation logic and integration contracts first
Manual regression with unknown coverage	Release decisions are based on opinion rather than evidence	Traceability matrix and coverage baseline in the first 4 weeks (Section 5)
All features tested equally	Low-risk areas consume effort that high-risk areas need	Risk scoring and tiered test depth (Section 3)
No central strategy or metrics	Each of the 8 teams works differently; quality cannot be compared	QA guild, shared metrics and release gates (Sections 7, 8)
Mixed legacy and microservices	Legacy code is hard to unit test, and service boundaries change often	Test legacy at its API boundary; protect services with contract tests
1 QA engineer to 2 developers per team; bi-weekly releases	QA capacity cannot cover manual regression of everything every two weeks	Developers own unit and contract tests; QA owns risk, strategy and higher-level testing

2.2 Assumptions (to be confirmed in the first two weeks)

The brief leaves some details open. This strategy assumes the following and states each assumption so it can be checked:

- The claim lifecycle is: first notice of loss (FNOL) → validation → coverage verification → assignment and assessment → calculation → approval → payment → recovery and close.
- Policy data lives in a legacy policy administration system that the platform queries during coverage verification.
- Third-party integrations include: independent assessors and repair networks (auto, property), medical providers and clearing houses (health), payment gateways and banks, fraud scoring, document capture/OCR, and regulatory reporting.
- Health claims involve protected health information and all products handle personal data, so privacy regulation applies (for example HIPAA or GDPR, depending on the market).
- Manual regression for a quarterly release takes roughly one week of combined QA effort. This figure becomes the baseline to improve against.

3. Risk Assessment

3.1 Scoring model

Each module, integration and business capability is scored as **Risk = Likelihood × Impact**, with each factor on a 1–5 scale. Scoring is done in a 90-minute workshop with QA, developers, product owners and a claims subject-matter expert, so the result reflects business impact and not only technical opinion.

Factor	What raises the score
Likelihood	Frequent change; complex business rules; legacy code; past incidents and defect density; dependence on external systems; no automated tests
Impact	Financial loss (over- or under-payment); regulatory or compliance breach; harm to the customer (delayed payment, wrong denial); claim volume affected; reputational damage

Scores map to three tiers: **Tier 1 (15–25)** critical, **Tier 2 (8–14)** significant, **Tier 3 (1–7)** low. The register is re-scored every quarter, and immediately after any Severity 1 or 2 production incident.

3.2 Initial risk register

The scores below are an initial view based on the incident pattern in the brief. The first workshop confirms or adjusts them.

Area	Example failure	L	I	Score	Tier
Health claim calculation	Deductible, co-pay or out-of-pocket maximum applied wrongly; coordination of benefits error	5	5	25	Tier 1
Auto claim calculation	Depreciation, total-loss threshold or salvage value miscalculated	4	5	20	Tier 1
Property claim calculation	Actual cash value vs. replacement cost, sublimits, catastrophe deductibles	4	5	20	Tier 1
Third-party assessor integration	Estimate imported with missing lines or wrong currency; status never updated	4	4	16	Tier 1
Legacy policy admin lookup	Coverage verified against an out-of-date or wrong policy version	4	4	16	Tier 1
Payment gateway / bank integration	Duplicate or missing payment after a timeout or retry	3	5	15	Tier 1
Medical provider / clearing house	Claim file rejected or mapped to the wrong procedure code	3	4	12	Tier 2

Area	Example failure	L	I	Score	Tier
Fraud scoring service	Service unavailable, so claims are auto-approved or stuck	3	4	12	Tier 2
Regulatory reporting	Late or incorrect statutory report	2	5	10	Tier 2
Authentication and roles (platform)	Assessor can approve their own estimate; data exposed to the wrong user	2	5	10	Tier 2
Customer FNOL portal	Claim submission fails on mobile; uploads lost	3	3	9	Tier 2
Document capture / OCR	Extracted values wrong; documents not attached to the claim	3	3	9	Tier 2
Agent workbench search and reports	Filters or exports incorrect	2	3	6	Tier 3
Notifications (email/SMS)	Status message delayed or wrongly worded	2	2	4	Tier 3

3.3 What each tier means in practice

	Tier 1 — Critical	Tier 2 — Significant	Tier 3 — Low
Test design	Full technique set: decision tables, boundary values, equivalence classes, negative and failure-mode tests	Main paths, key boundaries and the most likely negative cases	Happy path and smoke checks
Automation target	≥ 90% of business rules and contracts	≥ 70% of requirements	Smoke level; automate when cheap
Regression	Every commit (calculation, contract) and every release	Nightly and every release	Release smoke; full run quarterly
Exploratory testing	Chartered session each sprint on changed areas	Chartered session each release	Ad hoc
Non-functional	Performance, resilience and security tested each major release	Security scan each release; performance quarterly	Covered by platform-level scans
Release gate	Blocking: zero open Critical/High defects; QA and product owner sign-off	Blocking on Critical defects only	Non-blocking; tracked

This is where the effort comes from: Tier 3 areas stop receiving full manual regression every two weeks, and that time goes to Tier 1.

4. Test Strategy by Focus Area

4.1 Claims calculation accuracy

Calculation is the largest single source of incidents and has the highest financial impact. The aim is to test the calculation logic directly and exhaustively, independent of the UI.

- **Business rules as a catalogue.** Every rule (deductible order, co-pay, sublimits, depreciation schedules, total-loss threshold, rounding) gets an ID and a decision table agreed with claims experts and product owners in three-amigos sessions. The decision table is the test oracle.
- **Data-driven tests at the service/API level.** Each decision table row becomes a test case kept in a CSV or spreadsheet that business analysts can read and extend. One parameterized test executes all rows against the calculation service, so adding a rule means adding rows, not code.
- **Boundary value analysis.** For example: deductible met exactly or missed by 0.01; out-of-pocket maximum reached part-way through a claim; repair cost at exactly the total-loss threshold; payout at the coverage limit; policy effective-date boundaries and time zones; currency rounding.

- **Invariant (property-based) tests.** Generate thousands of random valid claims and check rules that must always hold: payout is never negative and never exceeds the limit; payout + deductible + member share = allowed amount; recalculating the same claim gives the same result.
- **Parallel run (golden master) against the legacy engine.** Replay a masked sample of historical claims through both the current and the new or changed calculation path, and compare the results to the cent. Every difference is triaged before release. This is the fastest way to cover legacy logic that has no documentation.
- **Product-specific packs.** Health: deductibles, co-insurance, out-of-network rules, coordination of benefits. Auto: depreciation, total loss, salvage, rental limits. Property: actual cash value vs. replacement cost, holdback release, catastrophe deductibles, sublimits.

Example: decision-table rows for an auto total-loss rule (assumed threshold: 75% of actual cash value)

Rule ID	Actual cash value	Repair estimate	Deductible	Expected outcome
AUTO-TL-01	20,000	14,999.99	500	Repair; payout 14,499.99
AUTO-TL-02	20,000	15,000.00	500	Total loss; payout 19,500.00 less salvage
AUTO-TL-03	20,000	22,000.00	500	Total loss; payout capped at actual cash value less deductible
AUTO-TL-04	20,000	400.00	500	No payout; below deductible; customer notified

4.2 Integration reliability

Integration failures are the other half of the incident problem. Many of them happen at boundaries the platform does not control, so the strategy combines contract testing (are we compatible?), virtualization (can we test without the partner?) and resilience testing (do we fail safely?).

- **Consumer-driven contract tests (Pact).** Every internal service-to-service call and every adapter in front of a legacy system or third party publishes a contract. Providers verify those contracts in their pipelines, and a "can-I-deploy" check blocks a release that would break a consumer. This catches breaking API changes between the 8 teams before they reach a shared environment.
- **Service virtualization (WireMock).** Each third party gets a virtual service that simulates success, validation errors, 4xx/5xx responses, timeouts, slow responses, malformed payloads and duplicate callbacks. Teams can then test failure paths on demand, which partner sandboxes rarely allow.
- **Resilience and idempotency.** Test retries, timeouts, circuit breakers and dead-letter queues. Payment calls must use idempotency keys: a test sends the same payment twice after a timeout and asserts exactly one disbursement.
- **Schema and mapping tests.** Validate inbound and outbound payloads against schemas, and test the field mappings (procedure codes, currencies, units, date formats) where many integration defects hide.
- **Partner sandbox end-to-end tests.** A small nightly suite runs the main flow against each partner's real sandbox to catch changes on their side.
- **Production reconciliation and synthetic checks.** A daily job reconciles approved claims against payment confirmations, and synthetic transactions exercise each integration every few minutes, so failures are detected in minutes, not when a customer calls.

4.3 Customer, agent and assessor journeys

- **Critical-journey end-to-end tests**, limited to about 10–15 per product. Examples: customer files a claim and uploads photos; agent triages, adjusts and approves; assessor receives an assignment and submits an estimate; customer tracks status through to payment.
- **Role and permission tests** at API level for all three user types, for example an assessor cannot approve a payment and a customer cannot see another customer's claim.
- **Usability and accessibility** for the customer portal: WCAG 2.1 AA checks, mobile and cross-browser runs, and exploratory sessions focused on stressed users filing a claim after an accident or loss.
- **Exploratory testing charters** for each Tier 1 and Tier 2 change, time-boxed and with findings recorded, so testers spend their time on what scripts cannot find.

4.4 Non-functional testing

- **Performance:** load tests modeled on catastrophe events, when property claims can rise to many times normal volume within hours. Includes surge FNOL submission, calculation throughput and third-party back-pressure.
- **Security and privacy:** OWASP-based testing and automated scanning in the pipeline; access control for personal and health data; no production personal data in test environments.
- **Resilience:** fault injection (for example Toxiproxy) in staging to confirm the platform degrades safely when a partner or legacy system is down.

4.5 Test data and environments

- Synthetic claim generators per product (policy, claimant, loss, estimate) let tests create their own data instead of relying on shared records.
- A masked, production-derived claim set for parallel runs and for realistic edge cases.
- Virtual services remove the dependency on unstable partner sandboxes for daily work, and containerized environments let each team test in isolation before the shared integration stage.

5. Coverage Plan

5.1 Making coverage measurable

1. **Inventory (weeks 1–3).** Collect every existing manual regression case and automated test. Map each one to a module, a business rule or requirement, and a risk tier. Mark duplicates and obsolete cases for retirement.
2. **Baseline (week 4).** Report requirement coverage by tier and by test level. This is the first time coverage becomes known, and the baseline every target is measured against.
3. **Traceability from then on.** Requirements and rules live in Jira; test cases live in a test management tool (for example Xray) linked to them; automated tests carry the requirement ID as a tag, so results flow back into coverage automatically.
4. **Definition of done.** A story is done only when its tests are linked and passing, at the level its tier requires.

5.2 Target coverage by area and test level

● = primary coverage at this level ◐ = partial or selective — = not applicable

Area	Unit	Calc / API	Contract	Virtualized integration	E2E UI	Exploratory	NFR
Claims calculation (all products)	●	●	◐	—	◐	●	◐
Third-party integrations	◐	●	●	●	◐	◐	●
Legacy policy admin	—	●	●	●	◐	◐	◐
Platform services	●	●	●	◐	—	◐	●
Customer / agent / assessor UI	◐	◐	—	—	●	●	●
Roles and permissions	◐	●	—	—	◐	◐	●

5.3 Regression strategy

Suite	When	Scope	Target duration
Smoke	Every deployment to any environment	Health checks; one claim per product end to end; integration heartbeats	< 10 minutes
Risk-based regression	Every bi-weekly release	All Tier 1 suites, plus Tier 2 suites for areas changed (chosen by impact analysis)	< 4 hours, mostly automated
Full regression	Quarterly major release	All tiers; parallel run; performance and resilience; exploratory sessions on Tier 1	2–3 days, down from about 1 week

6. Automation Recommendations

6.1 Order of investment

Automation is introduced in the order that removes the most incident risk per unit of effort:

1. Data-driven calculation tests for health, then auto, then property.
2. Contract tests on the five Tier 1 integration boundaries.
3. Virtual services for the payment gateway and the assessor network, including failure modes.
4. API regression suites for the product modules, which currently have none.
5. Parallel-run harness comparing the legacy and current calculation results.
6. A small critical-journey UI suite per product.

Target shape of the suite: about 60% unit, 25% service/API and calculation, 10% contract and integration, 5% end-to-end UI. UI tests are deliberately few, because they are slow and fragile, and the risks in this platform sit in logic and boundaries, not in screens.

6.2 Recommended tooling

Need	Recommendation	Reason
Unit and calculation tests	The team's existing stack (e.g. JUnit 5 / pytest) with parameterized tests	No new language to learn; decision tables run as data
Contract testing	Pact with a Pact Broker	Consumer-driven contracts plus a can-I-deploy gate across 8 teams
Service virtualization	WireMock	Simulates partners and failure modes; runs in CI containers
API and UI end-to-end	Playwright (API and browser)	One framework for both; fast, reliable, good tracing
Performance	k6	Scripted load tests that fit into the pipeline
Security	OWASP ZAP and static analysis in CI	Automated baseline on every build
Resilience	Toxiproxy	Inject latency and outages between services
Test management and traceability	Jira with Xray	Links requirements, tests and results for coverage reporting
Reporting	Allure reports and a Grafana dashboard	Shared view of results and metrics across teams

Where a team already has a working tool, it keeps it. Consistency matters most for contracts, reporting and test management, because those cross team boundaries.

6.3 Pipeline integration

Stage	Tests run	Time budget	Gate
Commit / pull request	Unit, calculation decision tables, consumer contract generation, static and security scans	< 10 min	Merge blocked on failure
Merge to main	Provider contract verification, API tests, virtualized integration tests	< 30 min	can-I-deploy check
Nightly	Critical UI journeys, partner sandbox tests, performance baseline, invariant tests	< 2 hours	Failures triaged next morning
Release candidate	Risk-based regression, exploratory sessions on Tier 1 changes	< 4 hours (automated)	Release gate (Section 7.2)
Quarterly release	Full regression, parallel run, load and resilience tests	2–3 days	Major release sign-off

6.4 Keeping automation trustworthy

- **Flaky-test policy:** a flaky test is quarantined within 24 hours and fixed or deleted within one sprint. Target flaky rate below 2%.
- **Shared framework:** the platform teams, which already have automation, own a common test framework and pipeline templates that the product and integration teams reuse.
- **Test code is production code:** it is reviewed, versioned and refactored like the application.
- **Developers write unit and contract tests**; QA engineers design the risk-based coverage, the calculation decision tables and exploratory charters, and pair on automation.

7. Metrics and Quality Gates

7.1 Quality metrics

Metrics are few and tied to outcomes. Counts of test cases or test runs are deliberately excluded, because they reward activity, not quality.

Category	Metric	Definition	Target (after 2 quarters)
Claims accuracy	Payout accuracy rate	Share of audited claims paid correctly (monthly sample)	≥ 99.5%
	Calculation defect leakage	Calculation defects found in production ÷ all calculation defects	< 5%
	Parallel-run differences	Unexplained differences between engines per release	0
	Claims reopened for payment error	Reopened claims ÷ closed claims	< 1%
Integration reliability	Integration incidents	Production incidents caused by integrations per release	−50% vs. baseline
	Contract coverage	Tier 1 integration boundaries covered by contract tests	100%
	Integration success rate	Successful third-party calls (after retries)	≥ 99.5%
	Mean time to detect	From failure to alert for integration issues	< 15 min
Customer experience	Claim cycle time	FNOL to payment, by product	−15% vs. baseline
	Customer-reported defects	Defects raised by customers or agents per release	−40% vs. baseline
	Critical-journey availability	Synthetic monitoring of FNOL, status and payment journeys	≥ 99.9%
	Post-claim satisfaction	CSAT survey after claim closure	Upward trend
Test effectiveness	Requirement coverage by tier	Rules and requirements with linked, passing tests	T1 ≥ 90%, T2 ≥ 70%
	Defect removal efficiency	Defects found before release ÷ all defects	≥ 95%
	Regression cycle time	Elapsed time for bi-weekly regression	< 4 hours
	Escaped defects by tier	Production defects per tier per release	Zero Critical in Tier 1

7.2 Release gates

Gate	Criteria
Bi-weekly release	All Tier 1 automated suites pass; no open Critical or High defects in Tier 1 areas; all contracts verified; risk-based regression complete; exploratory notes reviewed for Tier 1 changes

Gate	Criteria
Quarterly major release	All of the above, plus: full regression complete; parallel run with zero unexplained differences; performance within agreed limits at catastrophe-level load; security scan with no High findings; QA guild and product owner sign-off
Hotfix	Smoke suite plus the automated suites for the affected area; regression test added for the defect before closure

7.3 Reporting rhythm

- **Per sprint:** each team reviews its coverage and escaped defects in its retrospective.
- **Per release:** the quality dashboard is the basis of the go/no-go decision.
- **Monthly:** a quality review with engineering and product leadership, focused on trends.
- **Quarterly:** risk register re-scored, targets revisited.

8. Organization and Governance

- **QA guild:** the 8 QA engineers meet weekly, led by a test strategy owner (QA lead). The guild owns this strategy, the risk register, shared standards and the metrics dashboard.
- **Ownership by team type:** product teams own the calculation decision tables and tests for their product; integration teams own contracts, virtual services and partner sandbox tests; platform teams own the shared framework, pipelines and environments.
- **Whole-team quality:** with one QA engineer for two developers, developers write unit and contract tests and fix the build; QA engineers focus on risk analysis, test design, exploratory testing and coaching.
- **Business in the loop:** claims experts, underwriters or actuaries review calculation decision tables, because they are the real source of truth for what a correct payout is.
- **Release QA lead:** a rotating role per release coordinates cross-team regression and owns the release-gate checklist.
- **Incident feedback loop:** every production incident gets a root-cause review that answers "which test would have caught this?". That test is added and the area's risk score is updated.

9. Implementation Roadmap

Phase	Key activities	Exit criteria
0. Foundation Weeks 1–4	Risk workshop and register; regression inventory and coverage baseline; metrics baseline from incident history; QA guild formed; tools confirmed	Risk register approved; baseline coverage and metrics published
1. Highest risk first Weeks 5–12	Health and auto calculation decision-table suites; contract tests on the 5 Tier 1 boundaries; virtual services for payments and assessors; pipeline gates live; dashboard v1	Tier 1 calculation rules ≥ 60% automated; contracts gating releases; first release decided on the dashboard
2. Broaden Quarter 2	Property calculation suite; parallel-run harness; remaining integrations; API regression for product modules; critical-journey UI suites; catastrophe-load performance test	Tier 1 ≥ 90%, Tier 2 ≥ 70% coverage; bi-weekly regression < 4 hours; incidents trending to −50%
3. Optimize Quarter 3 onward	Test impact analysis to select tests by change; production synthetic monitoring and reconciliation; resilience testing; continuous risk re-scoring from incident data	Targets in Section 7 met and sustained across two quarters

10. Risks to This Plan

Risk	Mitigation
Limited QA capacity (1 per team) slows automation	Developers own lower-level tests; phase work by risk; reclaim Tier 3 manual regression time

Risk	Mitigation
Legacy systems are hard to automate	Test at the API or adapter boundary; use parallel runs instead of unit tests for legacy logic
Business rules are undocumented or disputed	Three-amigos workshops with claims experts; derive rules from historical claims and the parallel run
Partner sandboxes are unreliable or limited	Virtual services for daily work; sandboxes used only for the nightly check
Flaky tests erode trust in automation	Quarantine policy, flaky-rate metric, fixed ownership
Teams resist a central strategy	Start with the incident data, show early wins on Tier 1, let teams keep working tools

11. Conclusion

The platform's quality problem is one of focus, not effort. Effort is spread evenly while risk is concentrated in calculation and integrations. Scoring risk, automating those two areas first and measuring coverage and outcomes turns testing from a uniform manual activity into an evidence-based release decision. The first four weeks need no new tools, only a risk workshop and a coverage baseline, so the organization can start immediately and judge the results within the first quarter.

Appendix A: Sample Test Cases

Representative Tier 1 and Tier 2 cases showing the depth this strategy expects.

ID	Tier	Scenario	Expected result
TC-HLT-CALC-014	Tier 1	Health claim of 1,000; member has 100 of a 1,000 deductible remaining; 20% co-insurance	Member pays 100 + 180 = 280; plan pays 720; deductible marked met
TC-HLT-CALC-022	Tier 1	Claim pushes member past the annual out-of-pocket maximum	Member share stops exactly at the maximum; plan pays the rest; later claims in the year pay at 100%
TC-AUTO-CALC-007	Tier 1	Repair estimate exactly at the total-loss threshold	Vehicle declared total loss; payout = actual cash value − deductible − salvage
TC-PROP-CALC-021	Tier 1	Replacement-cost policy before repairs are complete	Initial payout = actual cash value; holdback released only after proof of repair
TC-INT-PAY-003	Tier 1	Payment gateway times out after accepting the request; platform retries	Same idempotency key used; exactly one disbursement; claim status "Paid"
TC-INT-ASR-009	Tier 1	Assessor estimate arrives with a missing mandatory line item	Message rejected to the dead-letter queue; alert raised; claim shows "Awaiting estimate"
TC-INT-POL-004	Tier 1	Policy endorsed mid-term; loss date falls before the endorsement	Coverage checked against the policy version in force on the loss date
TC-SEC-ROLE-005	Tier 2	Assessor attempts to approve a payment through the API	Request refused (403); attempt logged in the audit trail
TC-UX-FNOL-002	Tier 2	Customer files a claim on mobile with 10 photos on a slow connection	Submission completes or resumes; confirmation number shown; no duplicate claim